

剧本驱动机器人演示系统

项目分工、开发过程与训练记录

——基于剧本生成、仿真强化学习与嵌入式机器人执行的协同研发报告

文档类型	项目总结 / 项目分工 / 开发过程与训练记录
项目成员	朱清扬（前后端系统）、李奕霖（仿真强化学习）、杨骐泽（嵌入式机器人）
整理依据	三份项目技术文档与当前系统架构链路
形成日期	2026 年 4 月 22 日

说明：本报告重点记录项目分工、研发推进、联调过程与训练闭环，写作重心放在工程过程与系统协同，而非底层代码逐行展开。

摘要

本文档依据《微境剧场项目详细方案（软件部分）》《剧本驱动四舵机机器人实体演示系统技术文档》以及《基于实时舵机角度传输的机器人控制系统》三份材料整理形成，重点面向项目答辩、过程复盘和团队协同说明。报告围绕“前端输入文本—内容系统生成结构化剧本—Unity 仿真平台解析并执行—策略模型输出连续舵机角度—实体机器人完成动作演示”这一主链路，系统梳理三位成员在不同层级承担的职责分工、具体开发工作、阶段性问题、联调过程与训练记录。其中，朱清扬同学主要负责前后端内容生产系统与出站接口建设，李奕霖同学主要负责 Unity 本地服务、数字孪生仿真环境、PPO 强化学习训练与运行时推理闭环，杨骐泽同学主要负责机器人的机械结构、嵌入式主控、实时舵机角度传输与实体机器人执行通道。本文尽量从工程过程而非单点技术出发，展示本项目如何把创意输入、模型训练、硬件控制与真实演示整合成一条可落地、可复用、可持续迭代的完整产品链路。

关键词：剧本驱动；前后端系统；Unity；数字孪生；PPO；Behavior Cloning；Sim-to-Real；ESP32-C3；实时舵机角度传输；项目分工；开发记录

文档用途：可用于项目答辩材料补充、阶段成果留档、团队分工说明、开发过程复盘以及后续项目成员接手时的背景阅读。

目录

摘 要	2
一、项目概述	5
二、项目分工说明	5
2.1 阶段里程碑划分	7
三、总体开发过程	7
四、前后端系统开发过程	8
4.1 需求拆解与系统边界设计	8
4.2 前端工作台与交互流程建设	9
4.3 后端 API、数据层与状态管理	10
4.4 多智能体编排与内容生产链路	10
4.5 素材管理、配置中心与统一导出	11
4.6 阶段成果与个人贡献总结	11
五、仿真强化学习系统开发过程	12
5.1 运行中枢思路与技术选型	12
5.2 本地服务监听与剧本协议解析	13
5.3 数字孪生环境与状态空间设计	13
5.4 动作空间、奖励设计与自回归控制	13
5.5 示教数据、Behavior Cloning 预热与 PPO 优化	14
5.6 模型导出、Unity 部署与运行时推理	15
5.7 世界坐标融合、安全裁决与 Sim-to-Real 过渡	15
5.8 阶段成果与个人贡献总结	16
六、嵌入式机器人系统开发过程	17
6.1 机械结构与 3D 打印实现	17
6.2 硬件电路与主控系统搭建	18
6.3 网络通信与 HTTP 控制架构	19
6.4 实时舵机角度查询与设置机制	19
6.5 舵机驱动、FreeRTOS 任务与动作组织	20
6.6 白名单动作、真实验证与数据回流	21
6.7 阶段成果与个人贡献总结	21

七、跨模块联调过程记录.....22

八、训练记录.....23

 8.1 示教数据采集记录.....24

 8.2 Behavior Cloning 预热记录.....24

 8.3 PPO 强化学习训练记录.....25

 8.4 部署验证与运行时观察记录.....25

 8.5 真实机器人回流修正记录.....25

 8.6 训练记录的总体结论.....26

九、项目总结与后续展望.....26

一、项目概述

本项目面向桌面微缩剧场与具身智能演示场景，目标不是单纯做一个“会聊天”的前端页面，也不是只做一个“会动”的机器人底层控制器，而是构建一套从高层语义到低层执行的完整系统。整个项目把用户自然语言输入、剧本生产、角色场景提取、分镜组织、Unity 仿真、强化学习训练、真实机器人控制等多个原本分散的模块串联起来，形成了一个既能生成内容、又能执行内容、还能通过反馈继续优化的闭环架构。

从系统链路看，用户首先在前端工作台输入主题、对白、角色关系或表演需求；随后软件系统通过多智能体编排和阶段化内容生产机制，将创意逐步改写为带动作语义的标准剧本，并沉淀出角色、场景、镜头、图片、音频等结构化资产；之后，系统将这些资产组织为统一的 JSON 协议发送至 Unity 本地服务端；Unity 完成剧本解析、动作调度与运行时推理，结合数字孪生中训练得到的策略模型持续输出下一时刻四个舵机的目标角度；最后，嵌入式机器人通过 WiFi 局域网接收角度控制量，驱动真实四舵机机器狗完成相应表演动作。

因此，本项目的价值不在于某一个局部环节“技术很新”，而在于三个层级真正被接通：第一层是内容层，负责让用户把创意转化为可执行脚本；第二层是仿真与策略层，负责把脚本语义转化为连续控制逻辑；第三层是实体机器人层，负责把数字世界中的动作规划落地为真实世界中的机械运动。只有这三层同时成立，项目才具备完整的展示能力、研究价值与扩展空间。

二、项目分工说明

本项目采用“分层负责、接口协同、阶段联调”的组织方式。由于系统同时涉及前后端软件工程、强化学习仿真训练、Unity 运行时部署、嵌入式控制、机器人机构设计等多个方向，单一成员很难覆盖全部链路。因此，在开发初期，团队就按照系统天然分层进行了职责划分，使每位成员都能够围绕一个相对清晰的边界展开深入工作，同时又通过统一协议和联调机制保持整体一致。

具体而言，朱清扬同学负责项目的软件内容生产中枢，主要工作集中在前端工作台、后端 API、结构化数据管理、多智能体编排、剧本资产组织以及 JSON 出站接口建设；李奕霖同学负责 Unity 仿真平台与强化学习部分，包括本地服务监听、剧本协议解析、数字孪生环境设计、PPO 训练、Behavior Cloning 预热、模型导出与运行时推理闭环；杨骐泽同学负责实体机器人系统，包括机器狗机械结构实现、ESP32-C3 主控程序、HTTP 接口、FreeRTOS 任务调度、舵机 PWM 控制以及实时角度查询和设置机制。

这样的分工并不意味着三位成员彼此割裂。恰恰相反，本项目最关键的难点就在于分工之后仍能形成协同。比如，前后端系统输出的字段必须能被 Unity 可靠解析；Unity 生成的角度控制流必须与嵌入式端的接口格式、刷新频率和执行能力匹配；嵌入式端采集到的真实执行现象又需要回流到仿真训练和上层逻辑设计中。正是基于这种需求，项目过程中大量精力投入到了协议统一、接口约束、边界说明和跨模块联调上。

为便于项目答辩与后续成员接手，团队将职责划分汇总如下表。

成员	负责模块	核心职责	阶段性产出
朱清扬	前后端内容生产系统	前端工作台、后端 API、数据库模型、多智能体编排、角色场景分镜组织、素材管理、导出 Unity JSON	可交互创作界面、结构化剧本流水线、统一出站协议
李奕霖	仿真强化学习系统	Unity 本地服务、剧本解析、数字孪生环境、示教数据构建、BC 预热、PPO 训练、ONNX 导出、Sentis 运行时推理、安全裁决	训练环境、可执行策略模型、运行时调度与推理闭环
杨骐泽	嵌入式机器人系统	机器狗结构实现、ESP32-C3 控制程序、WiFi/HTTP 通信、FreeRTOS 任务、LEDC PWM、舵机角度查询与设置、真实动作验证	实体四舵机机器人、角度直控接口、真实执行通道

从表中可以看出，三位成员的工作分别对应内容层、仿真策略层与实体执行层。这种分工方式与项目系统架构高度一致，有利于成员围绕单一主线深入推进，同时通过统一接口完成集成。

2.1 阶段里程碑划分

为了避免跨学科项目在推进过程中出现目标漂移，团队还将整体研发分成若干阶段，每个阶段都对应明确的核心问题与责任主体。

阶段	主要目标	对应负责人	关键说明
阶段一：系统框架确定	统一总体链路和输入输出边界	三人协同	明确上层生成结构化剧本、中层 Unity 生成角度流、下层机器人执行动作
阶段二：子系统独立开发	各模块先形成可单独运行版本	按模块分工	前后端、仿真训练、嵌入式控制分别推进
阶段三：协议与接口收敛	让三个系统在字段和时序上可对接	朱清扬、李奕霖、杨骐泽	重点解决 JSON 结构、标签集合、角度范围、刷新频率等问题
阶段四：训练与部署验证	让策略模型真正参与执行闭环	李奕霖为主，杨骐泽配合	完成 BC 预热、 PPO 优化、模型部署和真机测试
阶段五：全链路展示完善	实现从文本到机器狗表演的完整演示	三人协同	补齐异常处理、素材组织、展示流程和答辩材料

三、总体开发过程

在总体推进上，项目并没有采取“先把所有模块各写一遍，最后再看能不能拼起来”的方式，而是采用了分阶段迭代路线。第一阶段解决系统边界和总体链路问题，明确三层结构：上层软件负责生成结构化内容，中层 **Unity** 负责把结构化内容转为可执行动作，底层机器人负责根据角度流执行动作；第二阶段分别在三个子系统内部开展深入开发，保证每一层先独立可运行；第三阶段开始协议收敛与双向联调，使数据能从上层顺利流向下层；第四阶段则重点处理训练、部署和真实执行之间的差异，让系统具备基本稳定性和展示能力。

在开发初期，团队首先统一了项目目标和输入输出形式。输入不再是简单按键，而是角色、台词、动作标签、约束信息组成的结构化剧本；中间表示不再是普通文案，而是可以落库、可编辑、可导出的角色表、场景表、分镜表和执行数据；最终输出也不只是

“播放一个动画”，而是由 Unity 在运行期持续生成的四舵机角度流。这个共识决定了后续所有模块的设计方向，也避免了成员各自从自己擅长的角度出发而导致接口不一致。

随后，三位成员分别进入各自主线。前后端部分围绕“如何让非专业用户也能完成内容创作并导出可执行资产”展开，重点解决分阶段界面、后端服务和结构化沉淀问题；仿真强化学习部分围绕“如何把高层剧本标签转成低层连续控制量”展开，重点解决状态设计、动作设计、奖励设计和训练闭环问题；嵌入式部分围绕“如何让真实机器狗稳定接收并执行角度流”展开，重点解决硬件结构、主控程序、通信协议和舵机控制精度问题。

当三个方向分别形成阶段成果之后，项目进入联调阶段。联调并不是简单地把三个模块首尾相接，而是要逐项核对字段命名、刷新时序、角度范围、异常兜底、动作中断策略以及真实机器人执行能力。正是通过这一轮又一轮收敛，系统最终形成了较为完整的从文本输入到机器人动作演示的闭环链路。

四、前后端系统开发过程

朱清扬同学负责的前后端系统，是整个项目的“内容生产与调度入口”。这部分工作的核心意义在于：让用户能够以较低门槛输入创意，并把模糊的自然语言想法逐步转化为后续仿真和机器人执行能够理解的结构化资产。如果没有这一层，强化学习模型和机器人控制系统就只能接收程序员手工编写的数据，无法体现项目“剧本驱动”的特点。

在具体开发目标上，这一部分既要面向用户体验，又要面向系统对接。一方面，前端界面必须清晰、分阶段、易操作，让用户能够看到项目、剧集、角色、场景、分镜、图片和音频等信息；另一方面，后端生成的内容必须具备严格的结构化特点，能够被数据库管理、能够反复编辑、能够以统一 JSON 导出，并与 Unity 侧约定字段稳定对接。因此，前后端部分不是简单的网页展示，而是一个真正意义上的内容中枢。

4.1 需求拆解与系统边界设计

在需求拆解阶段，朱清扬同学首先做的不是直接开始写页面，而是先明确软件部分到底承担什么、不承担什么。根据项目整体架构，软件系统承担的是认知与内容生产层：它需要理解用户创意、组织结构化剧本、提取角色与场景、拆分镜头、管理素材，并把

这些结果导出给执行层。相应地，机器人底盘控制、舵机驱动、物理安全机制和低层运动执行不应由这一层负责。这个边界定义非常重要，它保证了前后端设计始终围绕“内容资产”而不是“硬件细节”展开。

在完成边界划分之后，他进一步将软件系统拆成前端工作台、后端服务、数据库、智能体编排、素材生成与出站接口几部分。这样一来，前端负责交互与可视化，后端负责服务与状态流转，数据库负责结构化沉淀，智能体负责任务分工和内容生成，出站接口则负责与 Unity 对接。该拆分方式既符合软件工程中的分层思想，也方便后续把不同能力逐步补齐。

4.2 前端工作台与交互流程建设

前端部分采用阶段化工作台思路，而不是单一聊天框。其目标在于让用户看到整个内容生产流程，知道当前处于哪一阶段，上一阶段产出了什么，下一阶段又要补充什么信息。围绕这一思路，前端界面至少需要支持项目列表、剧集页、创作访谈入口、角色和场景管理、分镜展示、图片与音频管理、设置中心等多个工作区。这样设计的好处是，系统不再只是“生成一段文字”，而是把结果作为资产持续管理起来。

在交互设计上，朱清扬同学重点强调了分阶段推进与人工确认节点。一方面，创作访谈不要求用户一次给完全部信息，而是允许从一个主题、一句对白或一个冲突切入，由系统逐步追问并整理草案；另一方面，剧本改写、角色场景提取、分镜拆解等结果也并非一次生成后不可更改，而是允许用户检查、修订、重试和继续生成。这种人机协同式流程，使前端工作台更像一个内容生产控制台，而不是一次性输出文本的“黑箱”。

为了保证界面与后端状态一致，前端还需要展示任务状态、资源绑定状态和素材结果。例如，某一集是否已经生成格式化剧本，角色是否已完成音色分配，某一镜头是否已有参考图，TTS 是否生成完成等。通过这些状态设计，前端不只是展示结果，更是在帮助团队理解整个流水线当前运行到哪里。

4.3 后端 API、数据层与状态管理

后端部分的核心任务是把前端的交互请求转化为稳定的业务接口，并将中间结果持续写入结构化存储。围绕这一目标，朱清扬同学将后端服务划分为多个业务域，例如项目与剧集管理、创作会话、角色接口、场景接口、分镜接口、图片生成接口、音频生成接口、配置管理接口等。这样的拆分使不同对象都有相对清晰的读写入口，有利于后期维护与联调。

数据层的设计重点不在于追求复杂关系，而在于保证剧本相关对象都能被持续沉淀。项目有项目级信息，剧集有集级信息，角色、场景、镜头、图片、音频、生成任务也都应该有各自的数据对象和状态字段。这样一来，系统就能避免“大模型输出一段文字后就丢失过程信息”的问题，而是把每一次生成都纳入到可管理、可追踪、可复用的资产体系中。

在状态管理方面，后端还承担了流水线推进器的角色。也就是说，系统不仅保存结果，还知道当前一集已经进行到哪个阶段、是否允许进入下一阶段、某些配置是否已绑定、某些任务是否仍在异步处理中。这些状态对于后续的图片生成、音频生成和 Unity 导出都具有重要意义。

4.4 多智能体编排与内容生产链路

前后端系统的一个关键特点，是不是把全部生成任务都交给同一个通用模型一次完成，而是采用多智能体分工思路。朱清扬同学围绕这一原则，将创作访谈、剧本改写、角色场景提取、音色分配、分镜拆解、提示词生成等能力拆成多个相对专职的模块，使每一个模块都只关注一个高价值环节。这样做可以减少任务混杂带来的输出混乱，也便于人为在中间节点进行校正。

例如，创作访谈模块负责从用户输入中补全设定，让不完整的创意逐步沉淀为草案；剧本改写模块负责把草案整理成更标准、更适合后续消费的剧本文本；提取模块把剧本中的角色和场景信息抽出并做去重处理；分镜模块则把连续文本拆成镜头级结构，形成真正意义上的中间表示。通过这种链式编排，系统最终得到的不仅是“写好的故事”，更是一套可以被下游模块继续调用的内容对象集合。

这一链路的价值，在项目联调阶段体现得尤为明显。因为 Unity 执行层真正需要的并不是自由散漫的自然语言，而是能够指明角色、语句、场景、动作标签、节奏和约束的结构化输入。多智能体编排恰好把内容逐步收束为这样的格式，为后续仿真和控制端接入创造了条件。

4.5 素材管理、配置中心与统一导出

除了文本资产本身，软件层还需要管理图片、配音、参考图、镜头图等多模态素材。朱清扬同学在设计中将资源与剧集和分镜进行绑定，使某个镜头的图像、某个角色的音色、某段对白的配音都能被追踪和复用。这种设计的意义在于，项目展示时不仅有动作控制，还有更完整的叙事表现形式；同时，软件层也因此具备扩展到更复杂舞台演示的可能。

为了避免配置分散和运行时混乱，前后端系统还建立了集中式配置管理思想。文本模型、图片模型、音频模型以及部分智能体行为参数都不应该写死在代码中，而应当具备统一配置入口。这样做一方面有利于答辩或演示时快速切换方案，另一方面也为后续从单一实验原型走向可持续迭代的产品形态预留了空间。

在这一系列工作之上，最重要的一项产出是统一 JSON 导出能力。前后端系统不是终点，而是执行层的上游。因此，朱清扬同学最终把角色、场景、分镜、图片、音频等内容按统一组织方式导出，作为 Unity 本地服务的输入。可以说，这一导出协议就是三位成员协同工作的桥梁：没有它，软件层就无法把成果稳定交给仿真层。

4.6 阶段成果与个人贡献总结

从项目过程来看，朱清扬同学完成的不是某个孤立页面，而是一整套能够支撑内容生产闭环的软件基础设施。他的工作使系统具备了从创意输入到结构化资产生成再到对外导出的完整链路，也让“剧本驱动”这一概念不再停留在口号上，而是落实为前端可以操作、后端可以组织、执行层可以接入的数据流程。

更重要的是，这一部分工作为整个项目建立了上游标准。很多时候，跨模块项目失败并不是因为算法不够好，而是因为上游输出混乱、下游无法消费。通过对边界、接口、

状态和数据对象的清晰整理，前后端系统为后续的强化学习训练、Unity 运行时解析和实体机器人控制提供了稳定入口。这是项目能真正串起来的关键基础。

五、仿真强化学习系统开发过程

李奕霖同学负责的仿真强化学习部分，是整个项目从“会生成剧本”走向“会真实动作演示”的核心环节。这一部分的本质任务，是把上层剧本中的角色语义、动作标签和执行约束，转化为真实机器人在每一个控制时刻都能使用的连续舵机角度。也就是说，这一层不是简单播放预设动画，而是在 Unity 中建立一个既能训练又能部署的运行中枢。

从项目定位上看，Unity 在这里不是普通可视化工具。它一方面要运行本地 HTTP 服务，监听前后端系统发来的 JSON 剧本；另一方面要完成解析、排队、调度、模型推理、安全检查与对外发送。进一步地，为了让系统具备学习能力与泛化能力，还需要在 Unity 内部建立数字孪生训练环境，并利用 ML-Agents 进行示教学习与强化优化。正因如此，这一部分工作的复杂度非常高，既包含工程接入，又包含策略训练与部署。

5.1 运行中枢思路与技术选型

在技术选型阶段，李奕霖同学首先解决的问题是：为什么要以 Unity 作为中枢，而不是把训练、推理和显示分别放在多个系统中？原因在于，Unity 同时具备场景表达能力、物理仿真能力、训练环境承载能力以及运行时部署能力，能够把数字孪生、剧本调度和可视化演示放在同一框架内完成。这样不仅减少了上下游切换成本，也使训练环境与部署环境共享更多逻辑。

在强化学习方案上，项目选择了“示教初始化 + PPO 强化优化 + 真实回流修正”的路线。Behavior Cloning 用于让模型先学会基本动作形态和标签语义，PPO 用于让策略在奖励约束与环境扰动下学会更稳、更连贯的控制时序，而真实机器人执行结果则用于后续的域随机化和残差校正。这一组合路线比单纯的模板动作或单纯的强化学习都更适合本项目的四舵机低自由度机器狗场景。

5.2 本地服务监听与剧本协议解析

在开发过程中，李奕霖同学首先建立了 Unity 侧的本地服务入口，使其可以在运行时监听固定端口并接收来自前后端系统的结构化剧本。这个环节的目标并不是简单收一段字符串，而是要把 JSON 中的角色、语句、动作标签、节奏信息和约束条件解析成 Unity 内部可消费的任务对象。只有解析稳定，后续的调度和推理才有基础。

在解析设计上，需要考虑的不只是字段映射，还包括异常输入处理、缺省值兜底、任务队列组织和中断策略。例如，同一时刻是否允许插入更高优先级动作，若部分镜头缺失字段是否采用默认行为，若外部服务重复发送是否去重等。这些工作虽然不像训练算法那样显眼，却直接决定了系统在真实演示时是否会出现停顿、卡死或逻辑混乱。

5.3 数字孪生环境与状态空间设计

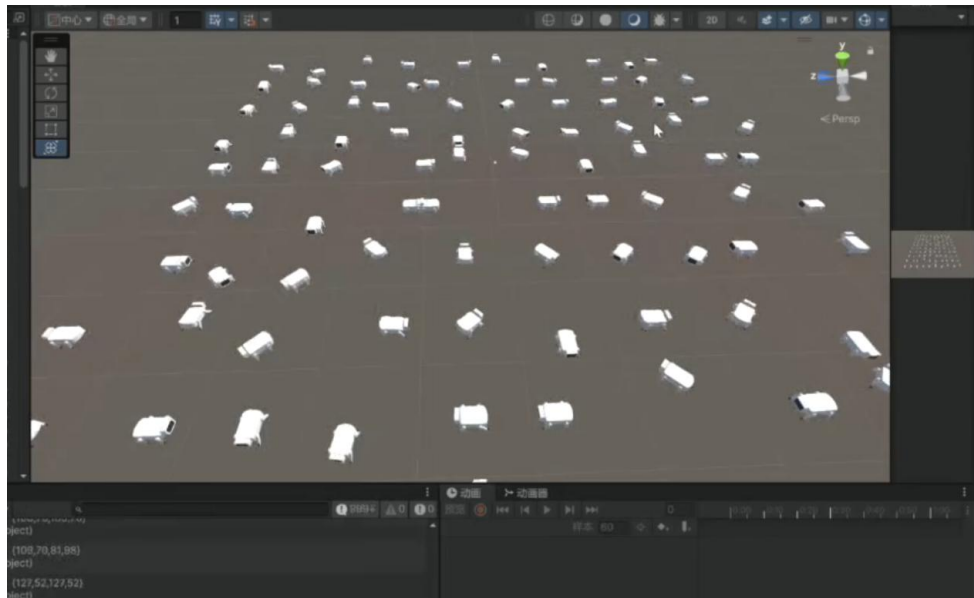
当本地服务入口建立后，下一步便是构建数字孪生环境。李奕霖同学在 Unity 中搭建四舵机机器狗的仿真场景，并将其抽象为可训练 Agent。在这一环境中，机器狗不仅要“动起来”，还要让观测、动作、奖励和终止条件能够形成标准强化学习闭环。通过这种方式，训练出的策略才有可能在运行时被真正调用，而不是停留在离线实验中。

状态空间设计是其中最关键的工作之一。由于系统目标不是纯步态学习，而是剧本驱动下的动作表达，因此状态不仅包括机器狗的位姿、关节状态、历史动作等信息，还需要包含当前动作标签、世界相机坐标、环境风险状态以及有限历史记录。换言之，策略输入中同时包含了“我现在在哪里、我刚才做了什么、剧本希望我接下来做什么、周围是否安全”四类信息。正是这种多源状态融合，使得系统具备了从高层语义向低层控制映射的能力。

5.4 动作空间、奖励设计与自回归控制

在动作设计方面，项目并没有采用“每个标签对应一段固定轨迹”的简化方案，而是让策略在每个控制周期输出四个舵机下一时刻的目标角度。这意味着，动作标签只是一个条件输入，真正执行的是连续生成的角度流。李奕霖同学围绕这一思想建立了自回归控

制框架：系统读取当前状态和历史信息，输出本时刻目标角度；执行后再读取新状态，进入下一个时间步。这样形成的不是僵硬切换，而是连贯推进。



奖励函数的设计则直接影响策略会学成什么样。为了避免模型只追求某一单项指标，奖励通常需要同时考虑多个维度，例如动作与剧本标签的一致性、姿态稳定性、前进方向正确性、能耗约束、越界风险、碰撞惩罚和跌倒惩罚等。在实际探索中，如果某个奖励项权重过高，模型就可能学出“为了不摔倒而不愿意动”或者“为了追求移动而动作失真”的问题。因此，奖励设计并不是一次定死，而是在训练观察中持续调整。

自回归控制还有一个工程价值，就是能够自然衔接真实运行时。因为真实机器人控制本身就是逐时刻发送角度目标，仿真中如果也按照这种方式学习，部署时就不需要再把模型输出额外转换成长轨迹片段，而是可以直接接入实体控制通道。

5.5 示教数据、Behavior Cloning 预热与 PPO 优化

为了避免强化学习从完全随机动作开始造成训练效率低、早期表现差的问题，李奕霖同学首先构建了示教数据。示教数据的来源主要包括 Unity 中的动作动画示教、模板动作回放以及围绕目标标签组织的专家轨迹。通过这些数据，模型可以先学会诸如前进、转向、问候、停顿等基本动作语义，使输出至少具备初始合理性。

在 Behavior Cloning 预热阶段，重点并不是追求最终最优表现，而是让模型初步建立“看到某种标签和状态后，应该朝什么方向输出角度”的关联。这个阶段通常能明显减少后续 PPO 训练中的无效探索。实际过程中，若示教数据分布不均或标签边界过于模糊，模型就会出现动作切换不清晰、输出抖动等现象，因此需要对数据进行适当平衡与清洗。

在完成预热后，PPO 训练阶段的目标是让模型从“会模仿”进化到“会在约束下自己优化”。PPO 的优势在于训练相对稳定，适合在 ML-Agents 框架中与环境并行采样、奖励反馈和策略更新结合使用。训练中，李奕霖同学重点关注的并不是单一损失数值，而是策略是否逐渐表现出更好的连续性、稳定性、标签响应能力和环境适应能力。

5.6 模型导出、Unity 部署与运行时推理

训练并不是终点，真正困难的是把训练结果稳定部署回运行时。为此，李奕霖同学进一步完成了模型导出与本地推理链路的设计。模型以标准格式导出后，在 Unity 侧通过本地推理库执行，使系统在不依赖外部云端推理的情况下，也能在演示现场完成实时决策。这一点对于竞赛和实体展示尤为关键，因为它显著降低了网络不确定性对演示效果的影响。

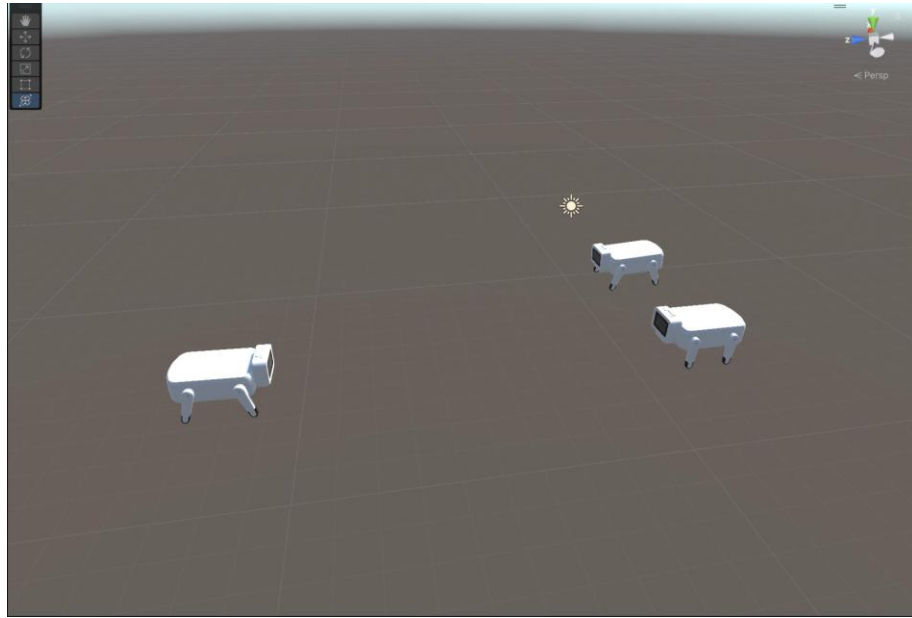
在部署阶段，需要重点处理训练环境与真实运行时之间的差异。例如，训练中观测到的状态向量如何在运行时准确拼装，历史记忆如何维护，推理频率如何与实体机器人可接受的控制周期对齐，若推理结果超出安全角度范围如何裁切等。这些问题看似实现层细节，实际上决定了模型能否真正“跑起来”。

5.7 世界坐标融合、安全裁决与 Sim-to-Real 过渡

由于项目最终面对的是实体机器狗，而不是只在虚拟空间中播放动画，因此安全问题必须在推理环节中考虑。李奕霖同学在方案中将世界相机坐标、风险信息和避障规则纳入到策略输入或安全裁决过程中，使系统具有最基础的环境感知能力。简单来说，模型不是在真空中生成动作，而是在知道自己大概位置和 risk 边界的情况下继续决策。

与此同时，Sim-to-Real 过渡也不能依赖“训练好就直接上真机”的想当然思路。仿真中的重力、摩擦、舵机响应和真实世界一定存在偏差，因此部署后往往会出现角度有效但

动作不协调、仿真平稳但真机抖动、长时间连续执行后偏差累积等问题。对此，李奕霖同学提出将真实机器人的白名单动作与仿真生成动作做对照，通过真实执行数据回流、域随机化和残差修正逐步缩小差距。这种做法比一次性追求“完美迁移”更加符合工程实际。



5.8 阶段成果与个人贡献总结

总体来看，李奕霖同学完成的是项目中最具“中台”性质的一部分工作。他既打通了前后端与机器人之间的执行中枢，又建立了训练环境、策略模型和运行时推理框架，使系统具备了从语义驱动到连续控制的核心能力。没有这一层，前端生成的剧本只能停留在文本层，嵌入式机器人也只能执行人工编写的固定指令，无法体现项目最关键的智能化特征。

更进一步地，这一部分工作还把项目从“能演示”提升到了“可研究、可优化”。通过示教预热、PPO 训练、部署验证和真实回流校正，系统逐步形成了闭环演化能力。也正因为如此，本项目不仅适合用于比赛展示，也具备后续继续深入扩展为多机器人、多动作自由度或更复杂交互系统的技术基础。

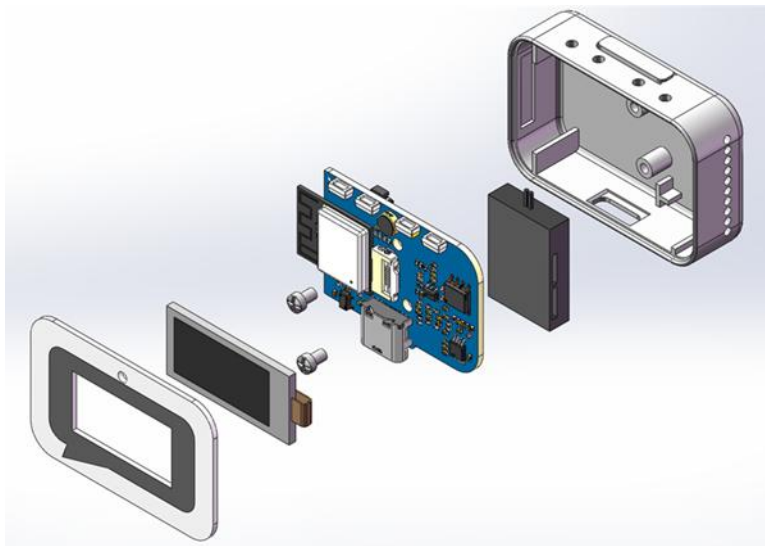
六、嵌入式机器人系统开发过程

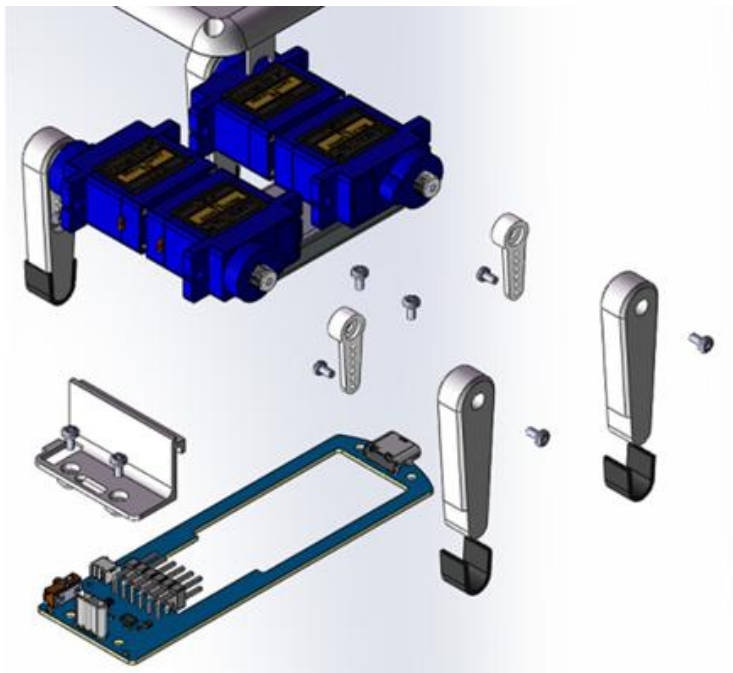
杨骐泽同学负责的嵌入式机器人系统，是整个项目真正落地到物理世界的最后一环。无论上层剧本设计得多完整、仿真训练做得多充分，若实体机器人无法稳定接收和执行控制量，那么整个项目依然只能停留在“看起来很完整”的软件演示层面。因此，这一部分工作的价值在于把数字世界中的控制结果变成真实机械运动。

与一般只依赖现成机器人底盘的项目不同，本项目的实体部分不仅包括程序，还包括机械结构、主控芯片、网络控制、舵机驱动和动作验证等多个层面。杨骐泽同学围绕“快速制造、实时控制、稳定执行、便于联调”这几个目标，对四舵机机器狗进行了从 3D 打印到网络控制的完整实现。

6.1 机械结构与 3D 打印实现

在实体机器人开发中，首先要解决的是机器狗本体如何快速从设计图纸变成可调试、可迭代的实物。杨骐泽同学采用了 3D 打印方案完成结构件制作，这种方式相较传统机械加工更适合竞赛型和研究型项目，因为它能显著缩短从设计修改到实体验证的周期。对于四舵机低自由度机器狗而言，结构件的精度、重量、安装孔位以及舵机装配方式都会直接影响最终动作表现，因此机械层面并不是可忽略的“外壳”。





在实际迭代中，3D 打印结构既要满足舵机安装与支撑，又要兼顾整体重心与动作稳定性。若腿部结构过重，舵机负担会增加；若连接件强度不足，长时间测试后容易松动；若尺寸设计不合理，则会导致装配偏差累积到控制层面。通过不断打印、装配、检查与修正，实体机器人逐渐具备了满足后续控制实验的物理基础。

6.2 硬件电路与主控系统搭建

在主控硬件选型上，项目采用 ESP32-C3 作为核心控制芯片。杨骐泽同学围绕这一芯片完成了 WiFi 通信、HTTP 服务、舵机控制和基础任务管理框架的搭建。ESP32-C3 本身兼具较强的网络能力和较低的开发门槛，适合本项目这种既需要局域网控制又需要实时舵机驱动的场景。

为了让机器人真正具备独立运行能力，硬件层不仅要连接四个舵机，还要处理供电、引脚映射、任务调度以及必要的显示或状态反馈功能。虽然从最终演示角度看，用户看到的只是“机器人动了”，但在底层实现上，每一路 PWM 输出、每一个引脚定义、每一次供电稳定性测试都会直接影响到动作是否抖动、是否偏移、是否能够长时间连续运行。

6.3 网络通信与控制架构

与许多仅支持串口通信或本地按键控制的简化机器人不同，本项目在嵌入式侧强调通过 WiFi 局域网建立统一的数据通道，使机器人能够真正接入上层仿真与控制系统，而不是作为一个孤立的硬件终端。杨骐泽同学在主控程序中完成了网络通信模块与通道管理逻辑的实现，使外部系统能够在局域网环境下与机器人建立持续连接，并围绕该通道完成目标角度下发、当前状态回传、链路检测和异常处理等关键功能。这样一来，Unity 运行时便能够以较低耦合度连接真实机器人，而不需要直接感知底层 PWM 驱动、舵机控制细节或嵌入式任务调度过程。

从工程实现角度看，本项目并未继续采用基于离散请求的 HTTP 控制方式作为主控制链路，而是选择建立一条面向实时控制的持久化数据通道。这样做的原因在于，情景剧场演绎系统中的机器人控制并不是低频、单次的命令触发问题，而是需要上层以固定周期持续发送控制量、下层持续回传执行状态的连续过程。相比请求—响应式接口，数据通道在控制频率、状态同步、时序一致性和链路持续性方面更适合这种实时演绎场景。通过这一方式，Unity、Python 测试程序以及后续其他调试工具都可以围绕统一的通信协议接入机器人系统，在不破坏底层实现封装的前提下实现联调、测试与部署。

从系统协同角度看，这种统一数据通道的重要意义，不仅在于“能够连上机器人”，更在于它为整个项目建立了一条稳定的中间桥梁：上层输出的是连续角度目标，中层传输的是结构化控制帧，下层执行的是经过约束与平滑处理后的舵机动作，同时机器人还能把当前状态再次回传给上层。也就是说，嵌入式侧不再只是被动接收几个固定动作命令，而是成为整个“剧本解析—策略推理—连续控制—实体执行”链路中的实时执行节点。对于多模块联合开发而言，这种通道化设计比单纯强调协议表面形式更有工程价值，因为它兼顾了可扩展性、实时性、可维护性和系统集成能力。

6.4 实时舵机角度查询与设置机制

杨骐泽同学完成的嵌入式部分中，最具创新意义的能力之一，就是实时舵机角度查询与设置机制。传统动作控制往往只接受“前进”“转向”“鞠躬”这类高层动作名，而真正怎么动全部固化在单片机内部。这样虽然使用简单，但无法适配上层强化学习模型生成的连

续控制量。本项目则进一步开放了单个舵机角度的实时控制接口，使系统能够直接发送四个舵机在当前时刻的目标值。

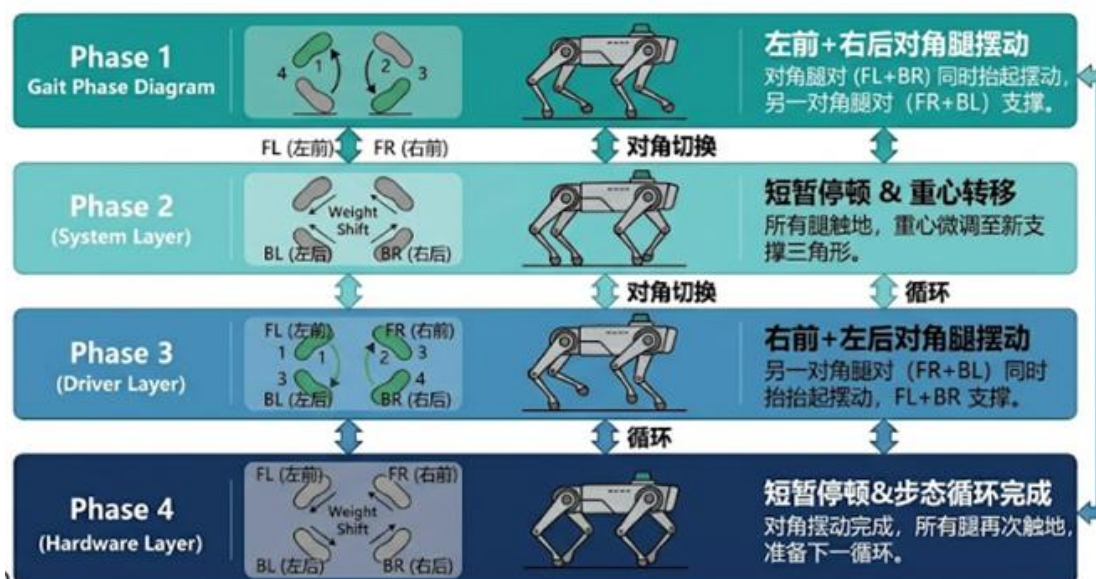
这一机制的意义极大。首先，它让机器人控制粒度从“整动作”下降到了“单关节”，从而能够接收仿真策略生成的细粒度结果；其次，它允许开发者实时查询当前角度，便于做校准、调试和闭环分析；再次，它把嵌入式部分从固定动作播放器转化为了通用执行终端，大大提升了系统的扩展性。可以说，若没有这一机制，仿真强化学习部分的连续控制思想就很难真正落到实体层。



6.5 舵机驱动、FreeRTOS 任务与动作组织

在更底层的实现上，杨骥泽同学围绕 LEDC PWM 输出、舵机角度映射和 FreeRTOS 任务调度构建了主程序框架。舵机控制看似只是把角度转为脉宽，但实际还涉及零位校准、范围限制、多个舵机同步刷新、动作过渡以及任务之间的协调。若这些问题处理不好，即使接口接收到的数字正确，机器人也可能表现为严重抖动、不同步或动作突变。

为提升系统可维护性，嵌入式程序还需要把不同职责拆分成相对独立的任务，例如网络通信、命令解析、动作执行、状态更新等。这样做的好处是，当上层发送新命令或需要查询状态时，系统不会因为单一阻塞流程而失去响应。同时，FreeRTOS 的任务模型也为后续扩展更复杂传感器或多种控制模式预留了空间。



6.6 白名单动作、真实验证与数据回流

虽然本项目最终强调实时角度直传，但实体机器人在开发和验证阶段仍然需要保留一批白名单动作作为参考。杨骐泽同学围绕前进、后退、转向、问候等标准动作完成了基本实现，使团队能够先验证机器人本体结构、主控系统和舵机驱动是否可靠。白名单动作还在后续训练回流中发挥了重要作用，因为它们能够作为“真实世界中已知可执行”的对照样本，为仿真策略提供参考基线。

在真实验证过程中，嵌入式层不仅检验动作能否执行，还暴露了很多只有上真机才会出现的问题，如舵机零位偏差、不同地面摩擦差异、电源波动导致的动作不一致、长时间运行后机构松动引发的误差累积等。正是这些反馈，为仿真层的域随机化和残差修正提供了重要依据，也让整个团队对 Sim-to-Real 差距有了更具体的认识。

6.7 阶段成果与个人贡献总结

总体而言，杨骐泽同学完成的是项目从数字闭环走向物理闭环所不可缺少的一环。他不仅做出了能运行的四舵机机器狗实体，还通过 ESP32-C3、WiFi 和 HTTP 接口，把这台机器狗变成了能够接受外部高层系统调度的通用执行终端。相比单独做一个遥控玩具式机器人，这种设计更符合项目作为具身演示平台的定位。

更重要的是，嵌入式部分为整个项目提供了真实性检验。很多在仿真中看似成立的控制思路，只有落到真实电机、真实结构、真实地面之后才能看出问题。通过实体机器狗的不断调试与执行，项目真正形成了“上层产生内容、中层生成控制、底层完成动作、真实结果再回流修正”的完整工程闭环。

七、跨模块联调过程记录

当三个子系统分别形成初步成果之后，项目进入最关键也最耗费精力的联调阶段。联调的难点并不在于某一段代码本身，而在于三层系统来自完全不同的技术栈：上层是前后端和内容生成逻辑，中层是 **Unity** 与强化学习运行时，下层是 **ESP32** 嵌入式控制与舵机驱动。若缺乏统一协议和边界约束，任何一层的小差异都会在联调中被放大。

因此，团队在联调过程中重点围绕三类问题展开收敛。第一类是数据问题，包括字段命名、层级结构、缺省值、动作标签集合和时间组织方式；第二类是时序问题，包括请求发送节奏、**Unity** 推理频率、机器人执行刷新频率以及动作中断策略；第三类是安全问题，包括角度范围裁切、异常数据兜底、执行失败回退和风险区域约束。只有这三类问题都被逐步解决，项目才能稳定地从软件侧一路跑到实体侧。

联调环节	主要参与成员	核心工作	解决结果
剧本字段收敛	朱清扬、李奕霖	明确角色、场景、分镜、动作标签与约束字段的组织方式	Unity 可以稳定解析上游导出的 JSON
运行时控制对齐	李奕霖、杨骥泽	对齐推理周期、角度范围、请求节奏与 HTTP 接口格式	仿真输出可被机器人持续接收与执行
真实执行反馈回流	李奕霖、杨骥泽	记录真机动作表现、抖动现象、偏差来源并回传训练侧	形成 Sim-to-Real 修正依据
整体流程演示	三人协同	从文本输入到机器人表演的全链路走通	形成可答辩、可展示、可复盘的系统闭环

在剧本字段方面，前后端系统一开始更偏向内容生产视角，而 **Unity** 执行层更关注哪些信息能被稳定消费。因此，团队需要把角色、场景、镜头、动作标签和执行约束做统一命名，并尽量减少歧义字段。通过这一过程，上游导出的 **JSON** 逐渐从“方便人看”转变为“既方便人看，也方便程序处理”的格式。

在时序对齐方面，强化学习运行时和嵌入式机器人之间需要不断试探一个合理平衡点。若 Unity 侧推理和发送过快，而机器人舵机本体来不及完成物理动作，就会表现为抖动和跳变；若发送过慢，则动作显得迟钝、不连贯。联调过程中，团队不断观察真机现象，在响应速度、动作平滑和接口稳定性之间寻找折中方案。

在安全兜底方面，系统最终形成了多层保护思想：上游输入阶段尽量约束动作标签和参数范围，Unity 侧在推理后进行裁切与风险检查，嵌入式侧再次对角度做底线约束。这样，即使某一层出现轻微异常，也不至于直接把风险传递到实体硬件。对于面向现场展示的系统而言，这种分层兜底非常必要。

八、训练记录

训练记录部分主要对应李奕霖同学负责的仿真强化学习链路，但其内容并不是孤立存在的。因为训练数据的来源、训练目标的设定、部署后的验证方式以及真实执行数据的回流，都与前后端系统和嵌入式机器人系统密切相关。因此，本节既记录训练阶段内部的主要工作，也说明训练如何与整体工程闭环结合。

需要说明的是，本项目的训练记录更偏重阶段性现象、问题定位和调整策略，而不是简单罗列若干漂亮曲线。原因在于，具身控制类项目真正重要的是策略是否在多轮实验中逐渐表现出更好的稳定性、连贯性、标签响应能力与真实部署适配性，而不是单次实验中某个数值是否达到表面最优。

结合项目训练闭环，可将训练记录概括为以下几个阶段。

阶段	训练目标	主要输入/方法	重点观察现象	调整方向
示教采集阶段	获得可模仿的专家轨迹	动画示教、模板动作回放、标签条件组织	样本是否覆盖主要动作、标签边界是否清晰	补足样本、平衡数据分布
BC 预热阶段	建立动作标签到角度输出的初始映射	Behavior Cloning 监督学习	是否能基本做出对应动作、是否存在抖动	清洗数据、优化标签组织

阶段	训练目标	主要输入/方法	重点观察现象	调整方向
PPO 强化阶段	在奖励约束下提升稳定性与连贯性	并行环境采样、PPO 更新	是否出现只求稳不愿动、或只求快而失稳	调整奖励权重、终止条件和动作裁切
部署验证阶段	让模型在 Unity 运行时稳定推理	ONNX 导出、Unity 本地推理	推理频率、历史状态维护、异常输出	修正输入拼装、对齐控制周期
真实回流阶段	缩小仿真与真机差距	白名单动作对照、真机执行记录、域随机化	零位偏差、摩擦差异、动作漂移	残差修正、物理参数扰动、再次验证

8.1 示教数据采集记录

训练工作的第一步并不是让强化学习直接从随机动作开始探索，而是先构建示教数据。示教数据主要围绕项目中需要频繁使用的动作标签展开，通过 Unity 中的动作模板、动画示教和专家回放，采集在不同状态条件下的角度变化序列。这个阶段的重点不是“做出最优控制”，而是给模型建立最初的动作语义感，让它知道某种标签大致对应什么姿态变化和时序趋势。

在采集过程中，一个明显的问题是不同动作之间的数据量往往不平衡。例如，前进类动作样本容易积累，而停顿、转向、问候等展示型动作如果采样不足，后续模型就会更倾向于学会常见动作，而忽视剧本表演中同样重要的表达型动作。因此，训练前期需要不断检查样本覆盖情况，避免示教集从一开始就把偏差固化进模型。

8.2 Behavior Cloning 预热记录

在获得初始示教集之后，项目进入 Behavior Cloning 预热阶段。此时模型的任务可以理解为“先学会模仿”，也就是看到状态和标签后，尽可能输出与专家轨迹相近的舵机角度。这个阶段通常能显著缩短后续强化学习的冷启动时间，因为模型不再需要从完全随机的空间里摸索基本动作。

从实验现象看，BC 预热往往能较快让模型学会大体方向，但也容易暴露连续控制中的一些问题，例如输出抖动、动作切换突兀、同一标签下时序节奏不稳定等。这些现象

提醒团队：示教数据除了要“有”，还要“干净且一致”。因此，在预热过程中，训练记录并不仅是保存模型版本，更包括对样本质量、标签粒度和状态拼装方式的持续检查。

8.3 PPO 强化学习训练记录

当模型具备初始动作能力之后，训练进入 PPO 强化阶段。此时的目标不再只是逼近专家轨迹，而是让策略在多种环境状态和奖励约束下学会更稳、更合理的动作组织方式。李奕霖同学在这一阶段重点关注几个现象：其一，模型是否能在保持动作语义的同时减少跌倒和失稳；其二，是否会出现为了避免惩罚而过度保守、不愿动作的情况；其三，是否能在不同标签切换下保持连续性，而不是频繁跳变。

训练中经常出现的一类问题是奖励偏置。例如，当稳定性惩罚过强时，策略可能学成“站着不动最安全”；而当前进奖励过强时，又可能出现动作幅度过大、执行不协调的问题。针对这些现象，团队并不是简单增减某一个参数，而是结合回放、观察和真实需求，对奖励项之间的相对关系进行重新调整。可以说，这一阶段的核心经验不是追求某个固定模板，而是不断逼近适合本项目场景的奖励结构。

8.4 部署验证与运行时观察记录

模型训练完成后，真正的考验是能否回到 Unity 运行时中稳定工作。部署验证阶段主要关注两件事：第一，离线训练时使用的状态输入能否在运行时正确还原；第二，推理输出在接入真实控制周期后是否仍然平稳可靠。为此，训练记录中不仅保留模型版本，也会同步记录部署时的输入拼装方案、推理频率和角度裁切规则。

在这一阶段，常见问题包括：运行时忘记维护某些历史状态，导致模型行为与训练期差异较大；推理节奏与机器人执行节奏不一致，造成动作断续；偶发异常输出未被裁切，影响展示稳定性。通过多轮部署观察与规则补强，系统逐渐形成了较完整的运行时推理闭环。

8.5 真实机器人回流修正记录

真实回流阶段是训练记录中极具工程价值的一环。因为仿真环境再完整，也不可能完全复刻真实舵机响应、零位误差、摩擦变化和机械结构松动等现象。因此，团队将真实

机器人中可重复执行的白名单动作作为参考，通过真机测试观察生成策略与参考动作之间的差异，并记录偏差类型。

这些偏差随后会反过来指导仿真侧的修正。例如，如果真机在某些角度区间内明显更容易抖动，就需要在训练或部署时加入更严格的裁切与平滑策略；如果某类动作在不同地面上的表现差异明显，就需要通过域随机化增强模型对环境变化的鲁棒性。通过这种方式，训练不再是一次性离线过程，而是逐步融入了真实世界的反馈。

8.6 训练记录的总体结论

综合来看，本项目的训练过程体现出一个非常明确的特点：训练不是为了替代工程，而是为了与工程共同组成闭环。示教数据帮助模型快速起步，PPO 帮助策略在奖励约束下自我改进，部署验证帮助发现训练与运行时之间的差距，真实回流又帮助修正仿真与真机之间的差距。正是这种多阶段、多来源的训练记录，使项目的强化学习部分更贴近真实可用系统，而不是停留在理想化实验。

九、项目总结与后续展望

通过本项目的开发，团队逐步完成了一条较为完整的技术闭环：朱清扬同学负责的软件系统解决了“用户如何把创意输入并转化为结构化剧本”的问题；李奕霖同学负责的仿真强化学习系统解决了“结构化剧本如何在 Unity 中解析、调度并转化为连续舵机角度流”的问题；杨骐泽同学负责的嵌入式机器人系统解决了“连续角度流如何在真实四舵机机器狗上被稳定执行”的问题。三位成员的工作各自独立又彼此支撑，共同构成了从文本到动作、从仿真到现实的整体方案。

从项目经验来看，最大的收获并不是某一个技术点本身，而是分层协同的工程方法。多模块项目如果没有清晰边界，往往会在后期陷入“谁都能改一点、谁也说不清到底该谁负责”的混乱状态；而本项目通过明确职责、统一协议、阶段联调和训练回流，较好地避免了这一问题。对后续继续扩展而言，这种组织方式同样具有价值。

面向下一步发展，本项目仍有很多可继续深化的方向，例如扩展更多动作自由度、引入更多环境感知信息、让剧本系统与更强语言模型深度协同、让实体机器人支持更高频

率和更精细的控制等。但无论如何扩展，目前形成的项目分工、开发过程与训练记录都已经说明：这套系统并不是几个孤立 Demo 的拼接，而是一条真实走通的、具备继续演化能力的技术路线。

结语：本报告中的内容按照三份技术文档的主线重新组织，突出团队分工、系统协同、开发过程与训练复盘，可直接作为项目材料的一部分使用。